Student Grade Management System

Overview

The Student Grade Management System is a Python application that allows users to manage classes, categories, assignments, students, grades, and grade calculations within a database. It provides a command-line interface for executing various functionalities ranging from student enrollment to grading.

Requirements

- Python 3.x
- MySQL Workbench (database)
- MySQL Connector
- Python libraries

Table Structure

Database schema students_grades_management contains the following tables:

`class`: Represents a Class and includes related information that has the following columns
|- class_id
|- term
|- course_number
|- section_number
|- class_description

IMPORTANT: In this table, term has to be entered specifically as below, otherwise the logic to get most recent class in ClassManagement will fail.

- 'Fall2023' for Fall of 2023
- 'Spring2023' for Spring of 2023
- 'Summer2023' for Summer of 2023

`category`: Defines different categories for grading within a class. This table has following columns.
|- category_id
|- weight
|- category_name
|- class_id

IMPORTANT: Weight has to be entered as a decimal value. For example, in my implementation I have entered the value as 25.00 if the category weight is 25%.

Also, please ignore if the output is displayed as Decimal('100') or something similar as this is how it is displayed in python (which was difficult to be changed) and the grades might not look proper due to the input values which are not customized accordingly, but it will work if done so in a proper manner.

`student`: Stores the information about Students that has the following columns.
|- student_id
|- username
|- first_name
|- last_name
|- email_address
|- address
`assignment`: Represents Assignments given in a class that has the following columns.
|- assignment_id
|- assignment_name
|- assignment_description
|- point_value
|- class_id
|- category_id
`grades`: Stores the Grades of each students' assignment that has the following columns.
|- student_id
|- assignment_id
|- grade
`enrolled`: Tracks the Enrollment information of students in particular classes that has the following columns.
|- student_id
|- class_id

Implementation and Execution

Possible Limitations

1. If term is not entered in the format described above, this code will not work to get recent terms based on Terms and may fail in other functionalities also.
2. My command line argument input doesn't accept the spaces between any input, for example while adding an assignment, below argument will fail

- `add-assignment Homework 1 Assignments 1st homework 80` Rather entering spaces between `Homework 1` and `1st Homework` please enter any special

characters such as `_ underscore` for the code to work. Try modifying the above instruction as below and it will work as expected:

- `add-assignment Homework_1 Assignments 1st_homework 80`

- **Installation of MySQL Connector for Python:**
  I used the `pip3 install mysql-connector` command to install the MySQL Connector for Python.
  This connector package provides the necessary tools and libraries for Python applications to interact with MySQL databases.
- **Connecting Python Code with MySQL Database:**
  With the MySQL Connector installed, I could easily establish a connection from my Python code to the MySQL database.
  The connector facilitated communication between the Python application and the database, enabling me to execute SQL queries, retrieve data, and perform database operations.
- **Development Environment in Visual Studio Code:**
  Visual Studio Code served as the development environment where I wrote, tested, and debugged my Python code.
  I leveraged Python 3.9 features and libraries along with the MySQL Connector to implement the functionalities required for managing student grades and classes as intended to do so.
- **Database Design in MySQL Workbench:**
  MySQL Workbench was instrumental in designing the database schema, defining tables, and establishing relationships between entities.
  I used the visual interface of MySQL Workbench to create an organized database structure that aligned with the requirements of my Python application.
- **Overall Workflow:**
  The integration between MySQL Workbench, Python 3.9, and Visual Studio Code provided a workflow for developing the students_grades_management application.

I wrote Python code to handle database operations, executed SQL queries, managed class enrollment, graded assignments, and calculated student grades effectively.
In summary, the MySQL Connector for Python, combined with MySQL Workbench for database design and Python 3.9 on Visual Studio Code for application development, resulted in a successful integration that facilitated smooth communication between my Python application and the MySQL database. My python code had the following classes along with the methods listed below:

1. **DatabaseManager Class:** Manages the database connection and provides methods for connecting to and closing the database.
   a. **get_connection():** Establishes a connection to the MySQL database.
   b. **close_connection():** Closes the connection to the MySQL database.

2. **ClassManagement Class:** Handles class-related functionalities such as creating classes, activating/selecting classes, displaying activated classes, and listing classes with number of students enrolled in each.
    a. **create_class(course_number, term, section_number, class_description):** Creates a new class in the database with the provided details.
    b. **random_address_generator():** Generates a random address for class-related purposes.
    c. **list_classes():** Retrieves and displays a list of classes from the database.
    d. **select_class(course_number, term, section_number=None):** Selects and activates a specific class based on the provided parameters.
    e. **show_class():** Displays the details of the currently active class.

3. **CategoryAssignmentManagement Class:** Manages categories and assignments, including showing categories, adding categories, showing assignments, and adding assignments.
    a. **show_categories():** Retrieves and displays the categories associated with the active class.
    b. **add_category(category_name, weight):** Adds a new category with the specified name and weight to the active class.
    c. **show_assignments():** Retrieves and displays the assignments associated with the active class.
    d. **add_assignment(assignment_name, category_id, description, points):** Adds a new assignment to the active class under a specified category.

4. **StudentManagement Class:** Manages student-related operations such as adding students, displaying students, and grading assignments for students.
    a. **add_student(username, student_id=None, first_name=None, last_name=None, email_address=None, address=None, class_id=None):** Adds a new student to the database or updates their information if they already exist. Can also enroll students in a class.
    b. **show_students(username=None):** Retrieves and displays student information based on the provided username or all students in the active class.
    c. **grade(assignment_name, username, grade):** Assigns a grade to a student for a specific assignment.

5. **GradeReporting Class:** Provides methods for reporting student grades and generating a gradebook.
    a. **student_grades(username):** Retrieves and displays the grades of a specific student based on their username.

    b. **gradebook():** Retrieves and displays the gradebook containing information about all students' grades in the active class.

6. **GradeCalculation Class:** Calculates grades based on student assignments and categories.
    a. **calculate_grades():** Calculates and displays the grades for each student in the active class based on assignments and categories.

7. **FinalApp Class:** Integrates all functionalities from the above classes and executes commands entered by the user through the execute_command method in python.
    a. **execute_command(command):** Executes various commands to interact with the database and manage class, category, student, and grade-related operations.

➢ Classes 2 to 6 inherits from the **DatabaseManager** class and the FinalApp class inherits from the other classes.

Instructions to run the code

To run the code provided, you can follow these steps:

- Ensure you have Python installed on your system.
- Copy the provided code into a Python script file, for example, grades_management_app.py.
- Make sure you have the necessary MySQL Connector Python package installed. You can install it using pip if you haven't already in the command prompt:

```
pip install mysql-connector-python
```

- Modify the database connection details in the DatabaseManager class according to your MySQL database setup (host, database name, user, password) and import the necessary python packages for the code to run efficiently as available from my implementation.

```
host='localhost'
database='students_grades_management'
user='root'
password='REPLACEME'
```

- Create an instance of the FinalApp class and start executing the commands from any editor that supports python, such as VSCode.